Lesson 2 Sidebars

Key Press Reactor

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

Changing Color: Tint, Swap, or Something Fancier?

Every engine in this lesson needed a way to change color per direction. Why did that one goal split into two completely different techniques - and which one would an actual studio ship?

There are two genuinely different families of technique for "make this sprite a different color," and this lesson ran into both. The first is color tinting (also called blending or modulating): multiply every pixel's color by a tint color, channel by channel. Godot's modulate, Unity's spriteRenderer.color, GameMaker's image_blend, and GDevelop's Tint color are all this same operation under different names. The second is sprite swapping: instead of recoloring anything, just show a completely different, pre-made image.

The catch with tinting: multiplying a pixel by a color can only ever darken or mix it - it can never brighten past what was already there. That means a tint only produces a clean, true color when the source art is white or grayscale to begin with (white times anything equals that color exactly). Tint a sprite that's already colored - like this course's placeholder art - and the result is a multiply of two colors, not a clean replacement. It comes out muddy, not wrong exactly, just not the color you asked for.

This course standardizes on sprite-swapping instead, for three reasons: it's the simpler concept for this age/skill range ("show a different picture" needs no explanation of multiply-blend math), it needed zero new assets beyond the color-per-direction art pack already being used, and it works identically in every engine with no exceptions to explain.

- **The one deliberate exception: TIC-80:** TIC-80's poke4() genuinely recolors in place, with no muddy-tint problem at all - because it isn't multiplying pixel colors together. TIC-80 draws using a small indexed palette (a fixed list of colors, each referenced by a number), and poke4() rewrites which actual RGB color a given palette index points to. Every pixel using that index instantly shows the new color, exactly, regardless of what the old color was. It's a genuinely different operation from tinting, not a simpler version of the same one - which is why it was kept as-is rather than switched to sprite-swap.

- **What Scratch is actually doing:** Scratch's "set color effect" (used for this lesson's Animate/blink goal, not Recolor) is a third category on its own - it cyclically shifts hue rather than multiplying against a tint color, which is why it doesn't have the muddy-on-colored-art problem either, but also doesn't let you land on one exact target color the way a true palette swap does.
- **What "the pros" actually do:** Real, shipped games use both families, deliberately. Palette-swap recoloring (TIC-80's approach, at a bigger scale) is a genuinely old, genuinely common trick - it's how a lot of classic games gave enemies recolored "palette swap" variants cheaply, and it's still how some modern games recolor uniforms/teams efficiently. True tinting is also used constantly, but professionally it's paired with source art built specifically for it: a white or grayscale "mask" texture made just for tinting, kept separate from the character's normal colored art, so the multiply always lands on an exact color. That's exactly the technique this course's tint/blend Super-Stretch idea invites a curious student to try by hand.

The Blink Nobody Coded the Same Way Twice

All 8 engines needed a sprite to blink while moving. All 8 used a genuinely different trick to do it. None of them is "the right way" - each is just the natural idiom for that engine.

Worth actually lining these up side by side, because the variety is the whole point:

- Scratch: there's no "key released" event to trigger off of, so blinking needs a polling forever-loop instead, using the Sensing "key pressed?" reporter plus a timer-mod trick ((timer mod 0.4) compared against half its own period) to get a repeating on/off flip.
- Godot: toggling the built-in visible property (inherited from every CanvasItem) on and off from an accumulating timer.
- Unity: the same idea, toggling SpriteRenderer's built-in enabled property from an accumulating timer.
- GameMaker: toggling the object's built-in visible variable, driven by a counter incremented in the Step event.
- GDevelop: no manual timer at all - a purpose-built Flash behavior ships with its own "Flash visibility (blink)" action.
- TIC-80: exploiting a documented, specific quirk of spr() - a scale factor of exactly 0 causes it to draw nothing at all - so alternating the scale argument between 0 and 1 makes the sprite blink with no second image and no visibility flag involved.
- Unreal: the same visible-toggle idea as Godot/Unity, but on an Actor rather than a sprite - Set Actor Hidden In Game, driven from Event Tick.
- 001 Game Creator: doesn't build a blink at all. It falls out for free, as an honest side effect of how this engine's movement already works - see below.

The 001 Game Creator case is worth slowing down for, since it's the most surprising one. This engine has no press-and-hold movement without extra work, so every "step" a student takes is really a full press-release cycle. The idle-reset script built for this lesson's Recolor goal fires on every release - including the ones in the middle of a walk, not just when a student actually stops. That reads as a blink while moving, and goes static the instant movement actually stops, purely as a side effect of how idle-detection was already built. It's explicitly documented as a deliberate, explainable consequence rather than a bug, and it's expected to disappear once true

press-and-hold movement is introduced (an existing Super-Stretch idea) - at that point Released will only fire on a genuine stop.

The takeaway worth naming out loud: "the right tool" genuinely depends on what a given engine exposes. A built-in behavior beats hand-rolled code when one exists (GDevelop). A documented rendering quirk can be repurposed cleverly (TIC-80). And sometimes the real "solution" isn't new code at all - it's noticing what code you already wrote is already doing (001 Game Creator).

Which Way Is "Up"? It Depends Which Engine You Ask

Move a sprite "up" in most engines this course uses and its Y value goes down, not up. That's not a GameMaker quirk - it's the majority convention. The real exceptions are worth knowing before they cause confusion.

There are two genuinely different coordinate conventions hiding across this course's 8 engines. Screen/canvas coordinates put the origin (0,0) at the top-left corner, with Y increasing downward - so moving "up" means subtracting from Y, not adding to it. Cartesian coordinates work the way a math-class graph does - an origin wherever makes sense, with Y increasing upward, the way it's normally drawn on paper.

GameMaker, Godot, and TIC-80 all confirmed use the screen/canvas convention (Y-down, origin top-left) in this course's own Lesson 2 movement code - GDevelop's built-in Top-down Movement behavior almost certainly works the same way under the hood, since it shares the same canvas heritage. The real exceptions are Scratch, whose origin is the CENTER of the stage with Y increasing upward (a deliberate, math-classroom-style choice), and Unity, whose world space is genuinely Cartesian and Y-up even in 2D, since Unity's "2D" is really a 3D scene viewed from one fixed angle.

Why the screen convention exists at all: it traces back to how a raster display draws an image, one row at a time starting from the top - the same convention nearly every image file format and 2D graphics API (including a web browser's own <canvas>) still uses today. GameMaker, Godot, TIC-80, and GDevelop all inherit that heritage directly. Scratch deliberately broke from it on purpose, aiming at people - often kids - who are more likely to have already met an x/y graph in math class than a raster display's internals.

- **Unreal is a third case, worth its own callout:** Unreal is also a full 3D Cartesian engine, but by convention it's left-handed with Z as the "up" axis, not Y, and X as "forward." That means moving between Unity and Unreal isn't just a sign flip - it's a genuinely different axis that means "up" in the first place. Worth flagging directly for any student working in both.

The practical habit worth building: before writing a single line of movement code in a new engine, check where the origin actually is and which axis - and which direction along it - counts as "up." Never assume it matches the last engine.

Scope: Where a Variable Lives, and How Long It Lasts

GameMaker's "is any direction key held" check turned into a lesson about how visual and typed code diverge for the same job. A completely separate bug, caught live while building a later lesson, turned out to be the same underlying question - variable scope - approached from the opposite direction.

Every variable in GameMaker - typed or built from GML Visual blocks - is really answering two questions the moment it's created: how long should this value stick around, and where does it actually live? GameMaker has exactly two answers, and picking the wrong one produces two very different, both genuinely common, failures.

An instance variable belongs to the instance itself. Create one - typically in the Create event, with a plain assignment and no special keyword - and it keeps whatever value it was last given for as long as that instance exists, visible to every event that instance ever runs afterward. A local (or "temporary") variable is the opposite: it belongs only to the single event or script call that creates it, and it's gone completely, with no trace, the instant that call ends. In typed GML, the difference is one keyword - var in front of a name makes it local; leaving var off makes it an instance variable (or updates an existing one). In GML Visual, it's two different actions instead of one keyword: Assign Variable creates or updates an instance variable, and Declare Temp explicitly creates a local one, scoped to just that one event.

Case 1: moving - correctly local, recomputed every Step

moving is a textbook case for local scope, and it's exactly what the code path already does: var moving = keyboard_check(vk_right) || keyboard_check(vk_left) || keyboard_check(vk_up) || keyboard_check(vk_down); declares it fresh, every single Step, with zero memory of what it was the Step before - which is exactly right, since moving is meant to be fully recomputed from the four key checks every time, never carried forward. An earlier version of this Sidebar's Visual Solution (below) used Assign Variable for this, which works - but Assign Variable's default is actually an instance variable, just one that happens to get overwritten every Step, not a true scope-for-scope match for the code path's local var. Declare Temp is the block that actually matches it: same intent, same lifetime, not just the same visible result.

Case 2: the opposite mistake - something that needs to persist, declared local by accident

The opposite case showed up for real, debugging a later lesson: a variable meant to accumulate or persist - a counter, a timer, a running total - needs the OPPOSITE choice from moving. It has to be created once, in the Create event, as a genuine instance variable, so each Step's changes actually stick. Declare it inside Step instead - or reach for Declare Temp out of habit, since "make a new variable" is exactly what it sounds like it does - and it gets wiped and recreated fresh every single Step, before whatever was just added to it has any chance to matter. The symptom looks exactly like "my increment isn't working." It IS working, for exactly one Step, and then getting quietly erased before the next Step even starts - which is a much harder bug to spot than an error message, because nothing actually breaks. The value just never seems to move.

- **The question worth asking before creating any variable:** Does this need to remember what it was a moment ago, or is starting fresh every time actually correct? If it needs to remember: create it once, in Create, as a plain instance variable (or Set Instance Variable, for a recognized built-in). If starting fresh is correct - or even the

whole point, the way it is for moving - Declare Temp (or var) is the right choice, and the more honest one: it also tells the next reader, at a glance, "this value never needs to outlive this one event," which a plain instance variable never says.

The tool-divergence story, underneath the scope question

GML Visual has no equivalent block for "wire four booleans into one OR." Its closest tool, If Key Down, is a branching If action, not a value you can chain - so a literal, block-by-block translation would mean building four brand new If Key Down checks solely to compute moving, right alongside the four If Key Down checks this lesson's movement/color logic already has for those exact same keys. Same condition, tested twice, in two places that now both have to agree forever.

The idiomatic Visual fix skips the duplication entirely: declare moving = false once, at the very top of the Step event, then drop one more action - Assign Variable, moving = true - directly into the branch of each of the four If Key Down blocks already handling movement and color for that direction. Nothing new to check, nothing checked twice, and every block on screen is still doing one clearly visible thing.

Worth naming directly: the text solution and the Visual solution aren't shaped differently because of a different design decision - nobody decided the Visual version should be more spread out. They diverge because of what each tool actually lets you build with. A single wired boolean expression is trivial to write and awkward to represent as blocks; reusing four branches that already exist is trivial visually and would be redundant if written out as text (nobody writing GML by hand would repeat the same assignment inside four separate if-blocks when one OR does it). The shape that ends up feeling "right" follows from the tool, not from any deeper truth about the underlying logic.

This also makes sense of something GameMaker warns about directly: converting a Visual event to typed code in the IDE is a one-way trip - once converted, that event can't cleanly convert back. That stops being surprising once the asymmetry above has actually been felt firsthand. Collapsing several blocks a person already arranged by hand down into one tidy expression is close to mechanical - just read off the structure that's already there. Going the other direction has no single obvious answer: does one compact OR expression become four new If blocks? Reuse existing ones? Something else entirely? Expanding text back into blocks would require the tool to guess at a structural decision a human hasn't actually made, which is exactly why it doesn't offer to.

- **The mid-path "cheat":** GML Visual's Value fields (Assign Variable's included) accept typed expressions directly - which means keyboard_check(vk_right) || keyboard_check(vk_left) || keyboard_check(vk_up) || keyboard_check(vk_down) can genuinely be pasted straight into a visual action's Value box, and it works. That's real mix-and-match between the two styles, and a legitimate, intentional GameMaker feature, not a hack. Worth being honest about what it actually is, though: a student using it isn't building an OR out of visual pieces - they're writing a line of GML text that happens to be sitting inside one field of an otherwise drag-and-drop project.

This is also not the first time this course has run into a version of the same idea. Lesson 2's own Animate (blink) goal used a genuinely different technique in all eight engines - not eight different design decisions, but eight different natural tools for "make this flicker" (a built-in visibility flag here, a documented rendering quirk there, a side effect of existing logic somewhere else - see "The Blink Nobody Coded the Same Way Twice," above). The Recolor goal hit the same pattern from the opposite direction: sprite-swapping was chosen specifically because it's the one

technique that behaves identically everywhere, sidestepping the fact that color-tinting's actual result depends on what art each engine's multiply math happens to land on (see "Changing Color," above). Line all four of these up - blink, color, the OR-check shape, and now scope - and the throughline is the same: the same underlying goal does not imply the same shaped solution, and the same variable name doesn't guarantee the same scope unless you actually chose it on purpose. What a tool exposes shapes the answer as much as the goal itself does - which makes it worth noticing, every time, whether a difference sitting in front of you is really a design difference, or a tool difference wearing a design decision's costume.

So: does the tool matter, if the job is the same?

Yes. Demonstrably, in this lesson alone, four separate times now.

The Visual Solution

Declare moving as a local (Declare Temp), false, once at the very start of the Step event - before any of the four movement checks run. This matches the code path's var moving = ... scope-for-scope, not just result-for-result. Then, inside each of the four existing If Key Down blocks (Right / Left / Up / Down) that already handle this direction's movement and color change, add one more action alongside what's already there - a plain Assign Variable is correct here, since moving already exists as a local for the rest of this event and this just updates it:

```
Step event (GML Visual):

Declare Temp: moving = false        <- local, recomputed fresh every step

If Key Down (Right) ->
  [this direction's existing movement + color actions]
  Assign Variable: moving = true

If Key Down (Left) ->
  [this direction's existing movement + color actions]
  Assign Variable: moving = true

If Key Down (Up) ->
  [this direction's existing movement + color actions]
  Assign Variable: moving = true

If Key Down (Down) ->
  [this direction's existing movement + color actions]
  Assign Variable: moving = true
```

No new condition checks, nothing tested twice, and moving ends up true if any direction is held and false if none are - exactly the same result as the one-line GML version, and now built with the block that actually matches its scope, not just its output.

Bounce Rewrites Direction - Three Different Places to Fix It

Scratch's "if on edge, bounce" does more than reposition a sprite - it changes its Direction, which can visibly flip or spin the costume. Three genuinely different fixes exist, and each one intervenes at a different point in the problem.

"if on edge, bounce" (Motion) is Scratch's purpose-built edge-collision block - one block instead of four manual boundary checks, and the block most Scratch tutorials reach for first. But its actual mechanism isn't just "nudge the sprite back onto the stage": it reflects the sprite's Direction as if it bounced off a wall, then repositions it along that new heading. Changing Direction isn't a side effect of the block - it's literally how the block works.

The visible symptom depends on the sprite's Rotation Style (see below): a costume set to "left-right" - the default for many sprites - visibly flips when Direction crosses past 90 or 270 degrees; a costume set to "all around" visibly spins to match its new heading. Either way, the sprite's actual reported Direction value really did change, on purpose, as the block's whole point.

Fix 1: Force it back

Keep "if on edge, bounce" exactly as-is, and immediately follow it with "point in direction [90]" (or whatever direction that script represents) to overwrite whatever Direction bounce just set. Bounce still does its real job - repositioning the sprite off the wall - but the very next block cancels the part of its output that wasn't wanted. This is the cheapest fix to add (one block, dropped right after bounce in each of the four key-press scripts) and it's the one that keeps "if on edge, bounce" itself in the project, which is worth something on its own: it's a block most Scratch tutorials reach for first, and it reads as native Scratch idiom in a way manual boundary math doesn't. The trade-off is conceptual, not practical: this fix operates on the result, deliberately overriding a value the block just set on purpose, rather than avoiding the situation that produced it.

```
when key [right arrow] pressed
change x by (10)
if on edge, bounce
point in direction (90)          <- overwrites whatever bounce just set
[color/sound/blink actions for this direction stay unchanged]
```

Fix 2: Clamp instead of bounce

Skip "if on edge, bounce" entirely. Instead, check the sprite's x position and y position directly against the stage's bounds each time it moves, and use "set x to" (not "change x by") to snap it back onto the edge if it's crossed the line - the same manual-clamp idea GameMaker's clamp(), GDevelop's Stay On Screen behavior, and Unreal's screen-bounds checks all reach for elsewhere in this course. Because this never touches Direction at all, there's nothing to override and nothing to flip - the fix works by never creating the problem in the first place. The cost is trading one clean built-in block for four separate boundary checks, one per edge, which is exactly why this course's Lesson 2 materials flag it as a later-lesson upgrade rather than the default Level 3 answer: it's the more correct fix, but also the more advanced one.

Fix 3: Turn off rotation for the sprite

A third option changes neither the block nor the script - it's a one-time, per-sprite setting instead. Click directly on the Direction field in the Sprite pane (the number next to the compass icon, above the code area) to open a small popup with three icons: all around, left-right, and don't rotate. Choosing don't rotate means the sprite's Direction value still changes exactly as before every time it bounces - nothing about the underlying data is different - but the costume itself never visually reflects that change, so there's nothing to flip or spin on screen. This is the cheapest fix in terms of ongoing work (set once, applies everywhere that sprite goes, no block added to any script), but it's also the bluntest: it silences ALL visual rotation for that sprite

permanently, including any place elsewhere in a project where the sprite is actually supposed to visibly turn to face its direction of travel.

- **Which one is "correct":** All three are legitimate, and they intervene at three different points in the same problem: Fix 1 corrects the result after the fact, Fix 2 avoids the cause entirely, and Fix 3 hides the visual consequence without touching the underlying value at all. None of them is wrong - which one fits best depends on whether a project needs Direction to be accurate for something else later (favoring Fix 2), whether "if on edge, bounce" specifically is worth keeping for its own sake (favoring Fix 1), or whether the sprite never needs to visibly rotate for any reason, ever (favoring Fix 3).